Iteration A3: Making the Page Update in Real Time

Real-time updates require WebSockets, JavaScript, and a bit of server code to make it all work. Fortunately, Rails has a feature called Hotwire that makes this easy.

First, we add the following line to the `app/models/product.rb` file:

rails80/depot_a/app/models/product.rb

```
  class Product < ApplicationRecord
    has_one_attached :image
»   after_commit -> { broadcast_refresh_later_to "products" }
  end
```

This line tells Rails to broadcast changes to the product model to any clients that are listening. We also need to add the following lines to the top of the `app/views/products/index.html.erb` file:

rails80/depot_a/app/views/products/index.html.erb

```
» <%= turbo_stream_from "products" %>
» <%= turbo_refreshes_with method: :morph, scroll: :preserve  %>
```

The first line tells the browser to listen for changes to the product model and update the page when they occur. The second line tells the browser to apply the changes directly to the page without refreshing it and to preserve the scroll position.

This is some pretty advanced stuff, and there is a lot more to the story.

- `after_commit` is an Active Record callback that's called after a record is saved to the database. We cover Active Record hooks in more detail starting with Chapter 7, *Task B: Validation and Unit Testing*.